

<https://medium.com/@TusharKanjariya/every-frontend-architecture-pattern-explained-a7ec07e7bbbb?sk=815c04e4979b3a452a9148bcae2181bc....>anlayse this medium article and explain me everything indetail so i can do in deep and understand this might help me in better understnading and interviews

Absolutely — **this is worth studying deeply for frontend interviews**, especially because you're already going through Next.js rendering, caching, App Router/Pages Router, SSR/SSG, etc. This article actually connects many of those concepts into a much bigger architecture picture.

I checked the Medium article you linked. The central idea is excellent:

Frontend architecture patterns are largely different ways of deciding **where the work happens — at build time, on the server, or in the browser.**  Medium

The article itself is relatively short (~10 minutes), so I **wouldn't recommend simply memorizing its patterns**. Instead, I'll use it as the starting point and build the *deeper mental model underneath it* — including things the article doesn't explain deeply enough.

## The way I want you to learn this

We'll turn this into a proper **Frontend Architecture Interview Track**, not just an article summary.

We'll cover:

1. What frontend architecture actually means
2. SPA architecture
3. MPA architecture
4. CSR
5. SSR
6. SSG
7. ISR
8. Server Components / RSC
9. Islands Architecture
10. Partial Hydration

11. Progressive Hydration
12. Streaming SSR
13. Edge rendering
14. BFF — Backend for Frontend
15. Micro-frontends
16. Module Federation
17. Monorepo vs polyrepo
18. Component-based architecture
19. Feature-based architecture
20. Layered frontend architecture
21. State architecture
22. Client/server state separation
23. API architecture
24. Caching architecture
25. CDN architecture
26. Rendering + caching combinations
27. How all of these fit together in Next.js
28. When to choose each architecture
29. Tradeoffs
30. Real-world interview questions
31. System-design questions
32. How to explain these in an interview

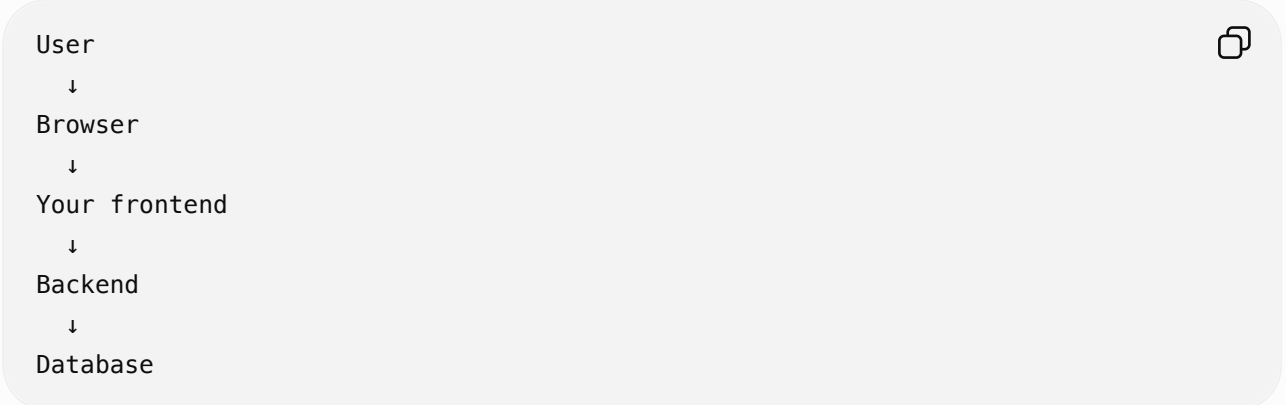
And importantly, we won't treat these as 30 unrelated buzzwords.

---

## Part 1 — The BIG mental model

Let's start with the idea from the article.

Imagine you have:



```
graph TD; User --> Browser; Browser --> Frontend[Your frontend]; Frontend --> Backend; Backend --> Database;
```

User  
↓  
Browser  
↓  
Your frontend  
↓  
Backend  
↓  
Database

The fundamental question is:

## Where should each piece of work happen?

For example:

### Option A — Browser does most of it



```
graph TD; Browser --> JavaScript; JavaScript --> API; API --> Database;
```



That's the basic idea behind **CSR / SPA**.

---

### Option B — Server does the rendering



```
graph TD; Browser --> Server; Server --> HTML; HTML --> Browser;
```



That's the basic idea behind **SSR**.

---

### Option C — Build server-generated HTML beforehand



```
graph TD; Build_time[Build time] --> HTML_generated[HTML generated]; HTML_generated --> CDN; CDN --> Browser;
```



That's **SSG**.

---

### Option D — Some UI is server-rendered, some is interactive

```
Server
├─ static UI
├─ dynamic UI
└─ interactive components
    ↓
  Browser
```



That's where things like **React Server Components / Islands / partial hydration** enter.



## But here's the important part

These aren't necessarily competing architectures.

This is where beginners often get confused.

For example:

"Is Next.js SSR or CSR?"

Wrong question.

Next.js can use:

```
SSR
SSG
ISR
CSR
RSC
Streaming
```



**within the same application.**

A single Next.js application can have:

```
/products
  → statically generated

/dashboard
  → dynamically rendered

/profile
  → server-rendered

/chat
  → client-heavy interactive UI
```



That's why we're going to build the concepts from first principles.

---

## 2. What does "frontend architecture" actually mean?

Before learning patterns, understand this distinction:

### Component architecture

asks:

How do I organize components?

Example:

Button  
Modal  
Navbar  
ProductCard  
ProductList



---

### Application architecture

asks:

How does my application work?

Example:

UI  
↓  
State  
↓  
API  
↓  
Backend



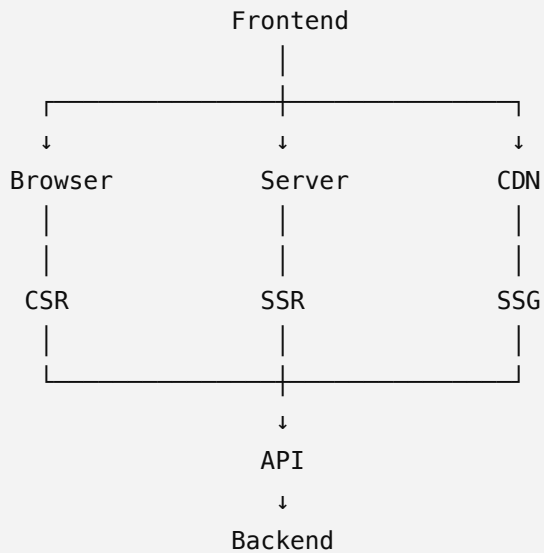
---

### Frontend architecture

asks a much bigger question:

How should the entire frontend system be structured and where should responsibilities live?

For example:



That's architecture.

### 3. The first major distinction: MPA vs SPA

This is one of the first things you should know for interviews.

#### MPA — Multi Page Application

Traditional websites.

Imagine:

```
google.com
  ↓
request /
  ↓
server
  ↓
HTML
```

Then you click:

```
/products
```

Browser makes another request:

```
GET /products
```

Server returns another HTML document.

So:

```
Page 1
  ↓
FULL PAGE REQUEST
  ↓
Page 2
```



Examples historically include traditional server-rendered applications.

---

## 4. SPA — Single Page Application

Now imagine React.

You initially request:

```
/
```



Server sends:

```
HTML
JS
CSS
```



JavaScript loads.

React takes control.

Then:

```
/products
```

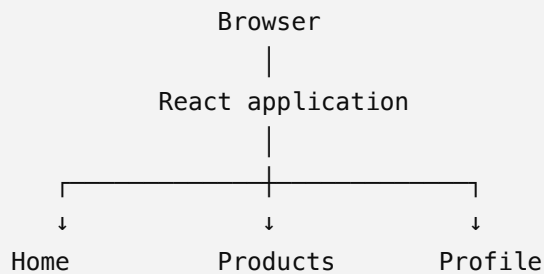


Instead of necessarily downloading another complete HTML document:

```
Browser
  ↓
React Router
  ↓
API
  ↓
Update UI
```



So the browser stays on essentially the same application shell.



That's the fundamental SPA model.

## 5. But SPA ≠ CSR

This distinction is **very important for your interviews**.

People often say:

SPA = CSR

They're related, but they're not the same concept.

### SPA

Describes the **application/navigation architecture**.

### CSR

Describes **where rendering happens**.

You can have:

SPA + CSR

but you can also have applications that combine client navigation with server rendering.

Think:

SPA  
↓  
Application architecture

CSR  
↓  
Rendering strategy

Different dimensions.

This distinction will become extremely useful when we reach Next.js.

## 6. CSR — Client-Side Rendering

Let's really understand this.

Suppose:

```
function App() {  
  return <h1>Hello</h1>  
}
```

With a client-rendered architecture, the browser gets something roughly like:

```
<div id="root"></div>
```

Then JavaScript loads:

```
app.js  
↓  
React  
↓  
App()  
↓  
<h1>Hello</h1>  
↓  
DOM
```



So:

```
Server  
↓  
HTML shell  
  
Browser  
↓  
JavaScript  
↓  
React  
↓  
render UI
```



## 7. Why CSR became popular

Because after the initial load, the browser can behave like an application.

For example:

```
Login
↓
Dashboard
↓
Settings
↓
Profile
```



You don't necessarily need a complete page reload each time.

Great for:

- dashboards
- admin panels
- highly interactive applications
- applications with lots of client state

---

## 8. But CSR has a cost

The browser has to do more work.

Something like:

```
HTML
↓
Download JS
↓
Parse JS
↓
Execute JS
↓
Fetch data
↓
Render UI
```



So the user may experience:

```
blank/loading
↓
JavaScript loads
```



```
↓  
data loads  
↓  
UI appears
```

This is one reason server rendering became important.

---

## 9. SSR — Server-Side Rendering

Now move rendering to the server.

Suppose:

```
GET /products
```



The server does:

```
request  
↓  
fetch products  
↓  
render React  
↓  
HTML  
↓  
browser
```



So the browser receives actual HTML:

```
<h1>Products</h1>  
  
<div>  
  Product A  
</div>  
  
<div>  
  Product B  
</div>
```

instead of an empty root.

---

## 10. Why SSR is useful

The browser can see meaningful HTML earlier.

That can help with:

- initial loading
- SEO
- content-heavy pages
- social previews
- slower devices
- slower networks

But there's a catch.

The browser still needs JavaScript for interactive components.

Which brings us to one of the most important concepts in modern React.

---

## 11. SSR does NOT mean "no JavaScript"

This is a **classic interview trap**.

Suppose:

```
<button onClick={handleClick}>
  Buy
</button>
```

Server can generate:

```
<button>Buy</button>
```

But HTML alone doesn't know:

```
handleClick()
```



The browser needs JavaScript.

So traditionally:

```
SSR
```

```
↓
```



```
HTML arrives
↓
JS loads
↓
React hydrates
↓
button becomes interactive
```

This leads us to...

## 12. Hydration

Hydration is basically:

**Connecting client-side React behavior to server-generated HTML.**

Think:

**Server:**

```
React
↓
HTML
```



**Browser:**

```
HTML
+
JavaScript
↓
React
↓
"Ah, I recognize this UI."
↓
attach behavior
```



Before hydration:

```
<button>Buy</button>
```



After hydration:

```
<button onclick="...">
```



Conceptually.

## 13. Why hydration can become expensive

Imagine a huge page:

Navbar  
Hero  
Product list  
Reviews  
Recommendations  
Footer  
Chat widget  
Analytics  
Comments



Server renders all of it.

Browser downloads JavaScript.

Then React hydrates a lot of the application.

Even if only this is interactive:

❤️ Like button



you might still have a large amount of JavaScript involved.

This led to newer architectural ideas.

---

## 14. Partial hydration

Instead of:

Hydrate everything



you try:

Hydrate only interactive pieces



Conceptually:

Server HTML  
|  
├─ Navbar      ❌ no JS  
├─ Hero        ❌ no JS



|             |   |       |
|-------------|---|-------|
| Article     | ✗ | no JS |
| Comments    | ✗ | no JS |
| Like button | ✓ | JS    |

This reduces client-side JavaScript.

## 15. Islands Architecture 🌴

This takes that idea further.

Imagine a page that's mostly static:

BLOG ARTICLE

Text...

Text...

Text...

[Interactive comments]

Text...

Text...

[Newsletter form]

The page is mostly HTML.

Interactive portions become **islands**.

HTML



Comments



Newsletter



Search

HTML

Hence:

Islands Architecture.

The idea is:

**mostly server/static HTML + small interactive client islands.**

## 16. React Server Components

Now we're getting into the territory that directly connects to your current Next.js studies.

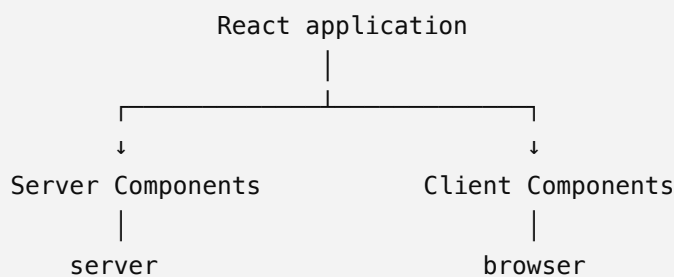
RSC changes the mental model again.

Instead of:

Everything → browser React



you can have:



Server Components can execute on the server.

Client Components are sent to the browser for interactivity.

## 17. The crucial distinction

### Server Component

Good for:

database access  
server-only secrets  
data fetching  
static UI  
reducing client JS



### Client Component

Needed for things like:

useState  
useEffect  
event handlers  
browser APIs  
interactive UI

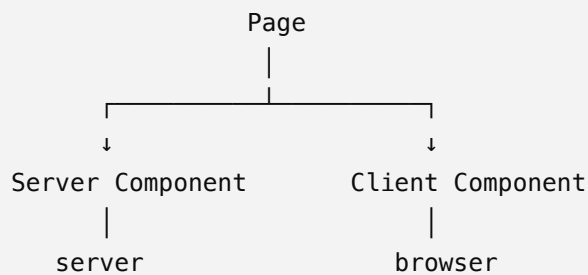


So instead of:

Entire page → browser



we can architect:



This is one of the biggest architectural shifts in modern React.

## 18. Now connect this to Next.js

This is where your current Next.js roadmap becomes much more meaningful.

You were learning:

Rendering  
Caching  
App Router  
Pages Router  
SSR  
SSG  
ISR  
RSC



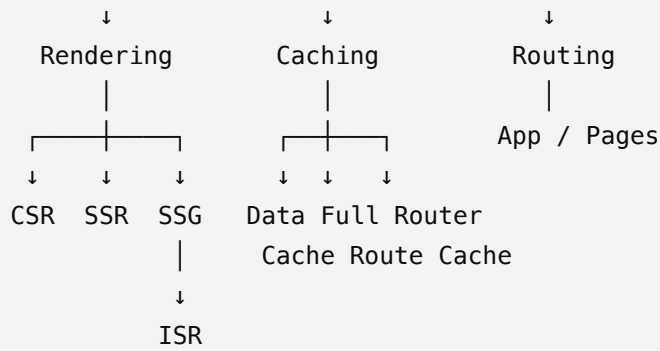
These aren't isolated Next.js features.

They're **architectural decisions**.

For example:

Next.js





And now the architecture article gives you the bigger context.

## 19. BFF — Backend for Frontend

This is another **very important interview concept**.

Suppose you have:

Web app  
Mobile app  
Admin app



And your backend has:

User service  
Product service  
Order service  
Payment service



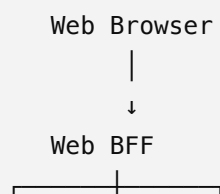
Without a BFF:

Browser  
├─ User API  
├─ Product API  
├─ Order API  
└─ Payment API



The frontend needs to understand lots of backend services.

With BFF:



↓      ↓      ↓  
User   Product   Order  
Service   Service   Service

The BFF is optimized specifically for that frontend.

For mobile:

Mobile  
↓  
Mobile BFF  
↓  
Backend services



So:

Web → Web BFF  
Mobile → Mobile BFF



---

## 20. Why BFF is useful

Imagine your product page needs:

product  
reviews  
inventory  
recommendations  
pricing



Without BFF:

Browser  
↓  
5 API calls



With BFF:

Browser  
↓  
GET /product-page  
↓  
BFF  
├─ Product  
├─ Reviews  
└─ Inventory



```
├─ Recommendations
├─ Pricing
↓
one frontend-friendly response
```

This can simplify frontend development and keep backend-service details away from the browser.

And **Next.js server-side code can sometimes play a BFF-like role**, but don't blindly call every Next.js API route a BFF. The architectural role matters more than the framework label.

## 21. Micro-frontends

Now imagine your company is enormous.

You have:

Amazon-like application



Different teams own:

Search  
Payments  
Orders  
Recommendations  
Account  
Seller dashboard

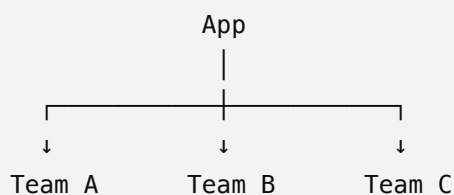


A single frontend repository might become difficult to coordinate.

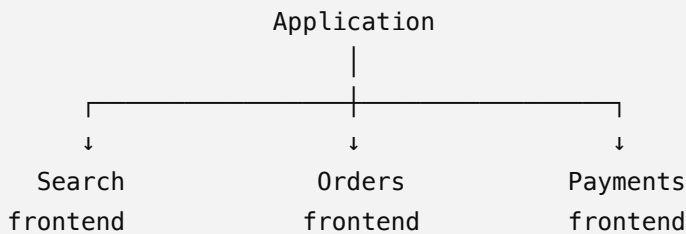
Micro-frontends try to apply the microservices idea to frontend ownership.

Instead of:

ONE GIANT FRONTEND



you may have:



Different teams can own different pieces.

## 22. But micro-frontends are NOT automatically better

This is another interview point.

You gain:

### Advantages

- Independent teams
- Independent deployments
- Team autonomy
- Technology boundaries
- Smaller ownership domains

But you pay:

- More complexity
- Deployment coordination
- Shared dependencies
- Performance concerns
- UI consistency
- Routing complexity
- State sharing
- Testing complexity

So don't say:

"Micro-frontends are better for large applications."

Say:

"Micro-frontends can be useful when organizational and deployment boundaries justify the additional frontend complexity."

That's a **much more senior answer**.

## 23. The architecture decision is really a tradeoff

This is probably the most important thing I want you to take from the article.

There is no:

🚫 1 Best frontend architecture.

There is only:

**Best architecture for a particular set of constraints.**

For example:

### Marketing website

You might choose:

SSG  
+  
CDN  
+  
minimal JS



---

### E-commerce product page

Maybe:

SSR / SSG / ISR  
+  
RSC  
+  
CDN  
+  
caching  
+  
small client islands



---

### Admin dashboard

Maybe:

SPA/CSR  
+



client state  
+  
API/BFF

## Huge organization

Potentially:

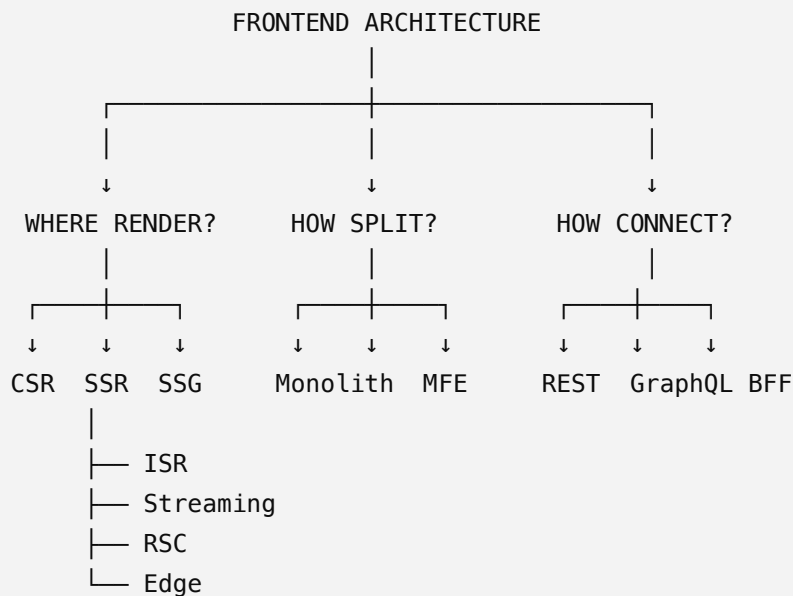
Micro-frontends  
+  
BFF  
+  
independent deployments



## 24. Here's the mental map I want you to memorize

Don't memorize definitions individually.

Memorize this:



Then another dimension:

HOW MUCH JS?



↓  
Everything  
|  
— SPA

```
|  
├─ SSR + hydration  
├─ Partial hydration  
├─ Islands  
└─ Server Components
```

And another:

HOW IS THE CODE ORGANIZED?



```
↓  
  
Component  
  ↓  
Feature  
  ↓  
Layer  
  ↓  
Domain  
  ↓  
Microfrontend
```

## 25. And this is where your Next.js roadmap should expand

Since you're already learning Next.js deeply, I would **add a dedicated Frontend Architecture chapter after the rendering/caching foundations.**

Something like:

NEXT.JS / FRONTEND ARCHITECTURE ROADMAP



```
|  
├─ 1. Browser fundamentals  
├─ 2. React fundamentals  
├─ 3. React rendering  
├─ 4. Next.js fundamentals  
├─ 5. Mutations  
├─ 6. Rendering  
|   ├── CSR  
|   ├── SSR  
|   └── SSG
```

- |
  - |— ISR
  - |— Streaming SSR
  - |— RSC
- |— 7. Caching
  - |— Data Cache
  - |— Full Route Cache
  - |— Router Cache
  - |— revalidation
- |— 8. Routing
  - |— App Router
  - |— Pages Router
- |— 9. Frontend Architecture 🌟
  - |— SPA vs MPA
  - |— CSR vs SSR vs SSG
  - |— Hydration
  - |— Partial hydration
  - |— Islands
  - |— RSC
  - |— Edge rendering
  - |— BFF
  - |— Micro-frontends
  - |— Module Federation
  - |— Monorepos
  - |— State architecture
  - |— API architecture
  - |— CDN architecture
  - |— Real-world system design

That would make your current Next.js learning **much more coherent**.

## 🎯 And yes — this is VERY useful for interviews

Suppose an interviewer asks:

"Why would you choose SSR over CSR?"

Don't answer:

"SSR is better for SEO."

Instead:

I would choose SSR when the initial HTML needs to contain request-specific or dynamic content and I want that content available before client-side JavaScript executes.



The tradeoff is increased server work and potentially higher time-to-first-byte, so I'd consider caching, CDN placement, streaming and whether the page actually needs server rendering.

That's architecture thinking.

Or:

"Why micro-frontends?"

Don't say:

"For scalability."

Say:

I'd consider micro-frontends when the primary scaling problem is organizational: multiple teams need independent ownership, deployment and release cycles.



I'd accept the additional runtime, dependency, integration and testing complexity only if that organizational benefit is worth it.

Much stronger.

## The BIG interview framework

Whenever an interviewer asks:

"Which architecture would you choose?"

Run these questions mentally:

### 1. Who needs the data?

Browser?  
Server?  
Both?



### 2. When can we generate it?

Build time?  
Request time?



After page load?

### 3. How dynamic is it?

Static  
Semi-static  
Dynamic  
Personalized  
Real-time



### 4. How interactive is it?

Mostly content  
Some interactions  
Highly interactive



### 5. How much JavaScript is acceptable?

Minimal  
Moderate  
Heavy



### 6. How many teams own the product?

1 team  
5 teams  
50 teams



### 7. How independently must things deploy?

Together  
or  
independently



### 8. What's the performance requirement?

SEO  
initial load  
interaction latency  
mobile network  
low-end devices



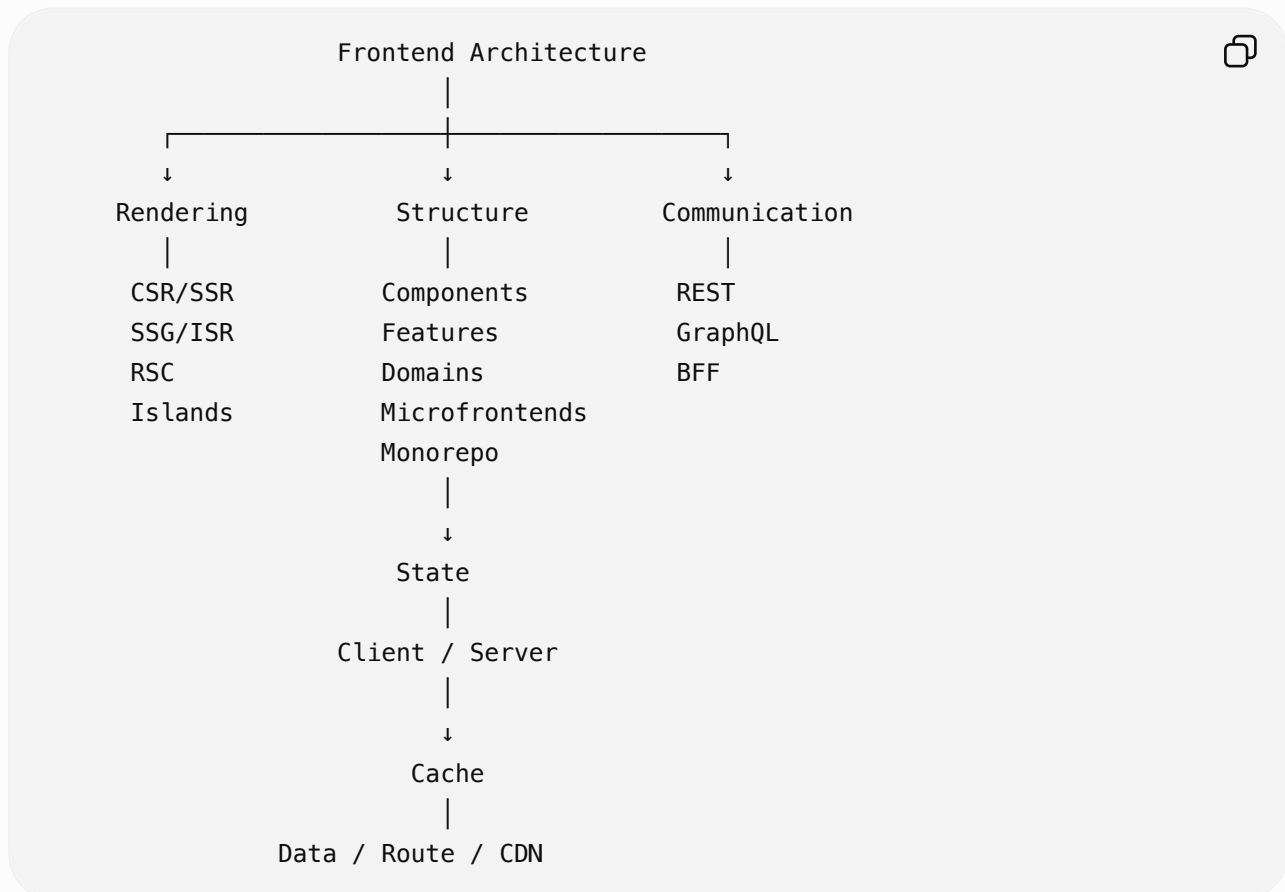
### 9. What's the operational complexity we can afford?

And **then** choose the architecture.

## One thing I want to change from the article

The article's central "browser vs server vs build time" idea is a **great starting point**, but don't let it become your entire definition of frontend architecture.

Architecture also concerns:



That's the deeper picture I'd like us to build.

And because you're already learning Next.js rather than just generic React, I'll keep tying every architecture concept back to **actual Next.js code and what happens under the hood** — which is exactly where these concepts become interview-useful rather than just theory. 🚀



wait first we will complete the nextjs part and project then we will come to this prt and will you cover all topics mentioned in the article ?

Yes ❤️ 100%. That's actually the better order.

Let's **not jump into frontend architecture yet**. We'll finish your **Next.js roadmap + project first**, and then come back to this article with all the foundation already in place.

And yes — when we come back, I'll cover **all the architecture patterns/topics mentioned in that Medium article**, not just give you a summary.

**Our order will be:**

### **Phase 1 — Finish Next.js deeply**

- Rendering fundamentals
- CSR / SSR / SSG / ISR
- App Router vs Pages Router
- Server Components / Client Components
- Streaming
- Suspense
- Caching
  - Data Cache
  - Full Route Cache
  - Router Cache
  - revalidation
- Mutations
- Forms
- Server Actions
- APIs / Route Handlers
- Middleware/proxy concepts
- Authentication/authorization
- Error handling
- Loading states
- Optimizations
- Deployment
- And the rest of your Next.js roadmap
- **Then build the project 🛠️**

### **Phase 2 — Frontend Architecture**

We'll return to this article and go **topic by topic**, including the article's concepts plus the deeper interview-level material behind them.

For every architecture pattern, I'll teach you:

**What → Why → How → Under the hood → Example → Tradeoffs → When to use → When NOT to use → Next.js connection → Interview questions**

And I'll explicitly tell you when something is:

- an **architecture pattern**
- a **rendering strategy**
- a **routing strategy**
- a **code-organization pattern**
- a **deployment pattern**
- a **communication/API pattern**

because these terms get mixed together *a lot* online.

## Most importantly

When we eventually reach this section, I won't just say:

"SPA means Single Page Application."

We'll go deep enough that if an interviewer asks:

**"Design the frontend architecture for an e-commerce application."**

you can actually reason your way through:

SPA / MPA?

↓

CSR / SSR / SSG / ISR?

↓

RSC?

↓

Hydration?

↓

Caching?

↓

BFF?

↓

State architecture?

↓

Micro-frontends?

↓

Deployment?

↓

Tradeoffs?



That's the level we're aiming for. 🔥

So for now, **park this article**. Don't study it separately.

We'll finish **Next.js** → **project** → then use **this article as the doorway into frontend architecture/system design**. That sequence will make much more sense. 🚀



ChatGPT can make mistakes. Check important info.